Nombre: Jaime Sebastian Esparza Leyva	**Matrícula**: 2097304
Nombre del curso: Componente de lógica de negocios	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: NA	**Actividad**: Unidad 5. Pruebas, integración y calidad
Fecha: 05/11/2026	
GitHub: https://github.com/Sebastian-Esp1/Unidad-5-CLN.git	

Contenido

Tabla 1. Contenido del informe

Sección	Contenido	Bloque	Página
1	Alcance y resumen: qué pide la unidad y qué se entrega	Introducción	3
2	Plan de pruebas y matriz de 16 casos (éxito, error y límite)	Entregable 1	5
3	E1: entorno virtual y pruebas del proyecto base	Evidencia	8
4	E2: pruebas unitarias del descuento VIP	Evidencia	10
5	E3: ciclo TDD con la regla "no confirmar dos veces"	Evidencia	13
6	E4: repositorio SQLAlchemy, commit y rollback	Evidencia	17
7	E5: pasarela de pago simulada con mock	Evidencia	19
8	E6: matriz de autorización de cancelaciones	Evidencia	21
9	E7: cobertura, Ruff y defectos detectados	Evidencia	23
10	E8: pipeline de GitHub Actions en verde, rojo y verde	Evidencia	27
11	E9: tres reglas de Sazón Urbano con 15 casos	Evidencia	31
12	Uso responsable de la IA	Reporte	37
13	Lista de verificación	Reporte	38
14	Reflexión del equipo	Reporte	40

1. Alcance y resumen

Este informe documenta la Unidad 5 (integración, pruebas y calidad) del Equipo 3. Parte del proyecto base entregado por el docente y lo amplía con una suite automatizada en pytest, pruebas de repositorio y de un servicio externo simulado, cobertura mayor al 80 %, un pipeline de GitHub Actions y la adaptación a tres reglas de Sazón Urbano.

Tabla 2. Resumen del proyecto

Campo	Contenido
Objetivo	Demostrar con pruebas automáticas que las reglas de negocio, la persistencia, la pasarela de pago simulada y la autorización funcionan, y que el pipeline rechaza cambios que rompen las pruebas.
Proyecto base	Proyecto_Python_Base_U5.zip: Order (descuento VIP 15 %), OrderService con pasarela de pago, repositorio SQLAlchemy y política de cancelación.
Herramientas	Python 3.11 o superior, pytest, pytest-cov, SQLAlchemy 2 con SQLite en memoria, Ruff y GitHub Actions.
Adaptación al PIA	Módulo sazon_urbano con tres reglas tomadas como referencia del paquete de datos del PIA. Es un ejercicio de esta unidad; el trabajo final del PIA se hace aparte.
Resultados verificados	205 pruebas pasaron, cobertura de 100 % sobre src/ (145 instrucciones), Ruff sin errores y 12 de 12 defectos introducidos a propósito fueron detectados. Se ejecutó con Python 3.13 y pytest 9.2 en Linux.

Tabla 3. Entregables solicitados y dónde están

Entregable	Dónde está	Sección
1. Plan de pruebas	docs/matriz_casos_prueba.csv (16 casos) y matriz de 15 casos de Sazón Urbano	2 y 11
2. Pruebas con pytest	tests/: dominio, repositorio, servicio, seguridad y Sazón Urbano	4 a 8
3. Integración con base de datos y servicios	test_repository.py, test_service.py, test_sazon_repository.py y test_sazon_service.py	6, 7 y 11
4. Cobertura mínima de 80 %	Reporte term-missing y htmlcov/index.html (100 %)	9
5. Pipeline de GitHub Actions	.github/workflows/ci.yml	10

Tabla 4. Hallazgos y correcciones al proyecto base

Hallazgo	Efecto	Corrección
pytest no encontraba la carpeta src con el comando pytest	El pipeline usa pytest a secas: la recolección fallaba con ModuleNotFoundError: No module named src (4 errores de colección).	Se agregó pythonpath = ["."] en pyproject.toml. Con el cambio pasan las 13 pruebas del base.
Ruff marcaba 12 errores	Con Ruff 0.17 (imports sin ordenar y datetime.timezone.utc) el paso ruff check del pipeline se detenía.	Se ordenaron los imports y se usó datetime.UTC.

Hallazgo	Efecto	Corrección
El manual y el laboratorio se contradicen	Matriz de 12 casos contra 9; cobertura en domain y application contra todo src/; cancelación del administrador antes de enviado contra SHIPPED nunca se cancela.	Se cumplió lo más exigente: 16 casos, cobertura sobre todo src/ y pruebas de lo que el código realmente implementa.

2. Plan de pruebas y matriz de casos

La matriz reúne los casos del proyecto base. La columna de resultado real se obtuvo ejecutando cada escenario contra el código; cada fila tiene su prueba automatizada.

Tabla 5. Matriz de casos de prueba

ID	Regla	Entrada	Resultado esperado	Tipo	Prueba	Resultado real
M01	Descuento VIP 15 %	Pedido 100, VIP	cobro 85, CONFIRMED	Éxito	`test_vip_discount`	cobro 85, CONFIRMED
M02	Cliente regular sin descuento	Pedido 100, regular	cobro 100, CONFIRMED	Éxito	`test_regular_no_discount`	cobro 100, CONFIRMED
M03	Descuento VIP 15 %	Pedido 200, VIP	cobro 170, CONFIRMED	Éxito	`test_vip_discount_total_200`	cobro 170, CONFIRMED
M04	Cliente regular sin descuento	Pedido 200, regular	cobro 200, CONFIRMED	Éxito	`test_regular_customer_total_200_has_no_discount`	cobro 200, CONFIRMED
M05	Cobro aprobado (mock)	Pedido 100, pasarela responde True	cobro 100	Éxito	`test_gateway_approved`	cobro 100
M06	No confirmar con deuda	Pedido 100, deuda 50	CustomerHasDebt	Error	`test_customer_debt`	CustomerHasDebt
M07	Total debe ser positivo	Total -1	ValueError	Error	`test_invalid_total[-1]`	ValueError
M08	Cobro rechazado (mock)	Pedido 100, pasarela responde False	RuntimeError: payment rejected	Error	`test_gateway_rejected`	RuntimeError: payment rejected
M09	No confirmar dos veces	confirm() dos veces	ValueError: already confirmed	Error	`test_cannot_confirm_twice`	ValueError: already confirmed
M10	Total debe ser positivo	Total 0	ValueError	Límite	`test_invalid_total[0]`	ValueError
M11	Total mínimo válido	Total 0.01	cobro 0.01, CONFIRMED	Límite	`test_smallest_positive_total_is_valid`	cobro 0.01, CONFIRMED

ID	Regla	Entrada	Resultado esperado	Tipo	Prueba	Resultado real
M12	Dueño cancela solo en las primeras 24 h	Dueño, estado NEW, exactamente 24 h	Denegado	Límite	`test_owner_exactly_24h_is_denied`	Denegado
M13	Administrador puede cancelar	Admin, pedido NEW de otro usuario	Permitido	Éxito	`test_admin_can_cancel_new_order_of_another_user`	Permitido
M14	Un pedido enviado nunca se cancela	Admin, estado SHIPPED	Denegado	Error	`test_shipped_denied_even_admin`	Denegado
M15	Persistencia con commit	Guardar pedido id=2, total 250	id=2, total=250, estado=NEW	Éxito	`test_repository_commit_order_250`	id=2, total=250, estado=NEW
M16	Dueño cancela solo en las primeras 24 h	Dueño, estado NEW, 23 h 59 min	Permitido	Límite	`test_owner_just_before_24h_is_allowed`	Permitido

Tabla 6. Distribución de los casos

Tipo	Requerido	Incluidos	Observación
Éxito	5	7	Descuento VIP y regular, cobro aprobado, administrador y persistencia
Error	4	5	Deuda, total negativo, cobro rechazado, doble confirmación y enviado
Límite	3	4	Total 0, total 0.01, 24 h exactas y 23 h 59 min

Las pruebas se clasifican en tres tipos. En el código las unitarias y las de integración están marcadas y pueden ejecutarse por separado. De las 205 pruebas, 189 son unitarias y 16 de integración; las de E2E no aplican en esta unidad.

Tabla 7. Tipos de prueba del proyecto

Tipo	Qué prueba	Archivos	Cómo ejecutarlas o nota
Unitarias	Reglas de negocio sin base de datos ni servicios externos. Marca: @pytest.mark.unit.	test_order.py, test_security.py, test_sazon_urbano.py	`python -m pytest -m unit -v`
Integración	El código junto con SQLite en memoria o con la pasarela de pago simulada (mock). Marca: @pytest.mark.integration.	test_repository.py, test_service.py, test_sazon_repository.py, test_sazon_service.py	`python -m pytest -m integration -v`
Extremo a extremo (E2E)	Recorrido completo de un usuario desde la interfaz hasta la base de datos.	No aplica	`Esta unidad no tiene interfaz ni API que recorrer; el trabajo se limita a la lógica de negocio, su persistencia y sus servicios.`

3. E1 · Entorno y pruebas del proyecto base

Se prepara un entorno reproducible y se ejecuta pytest sobre el proyecto. Todos los comandos se ejecutan desde la raíz del proyecto en la terminal de VS Code (PowerShell).

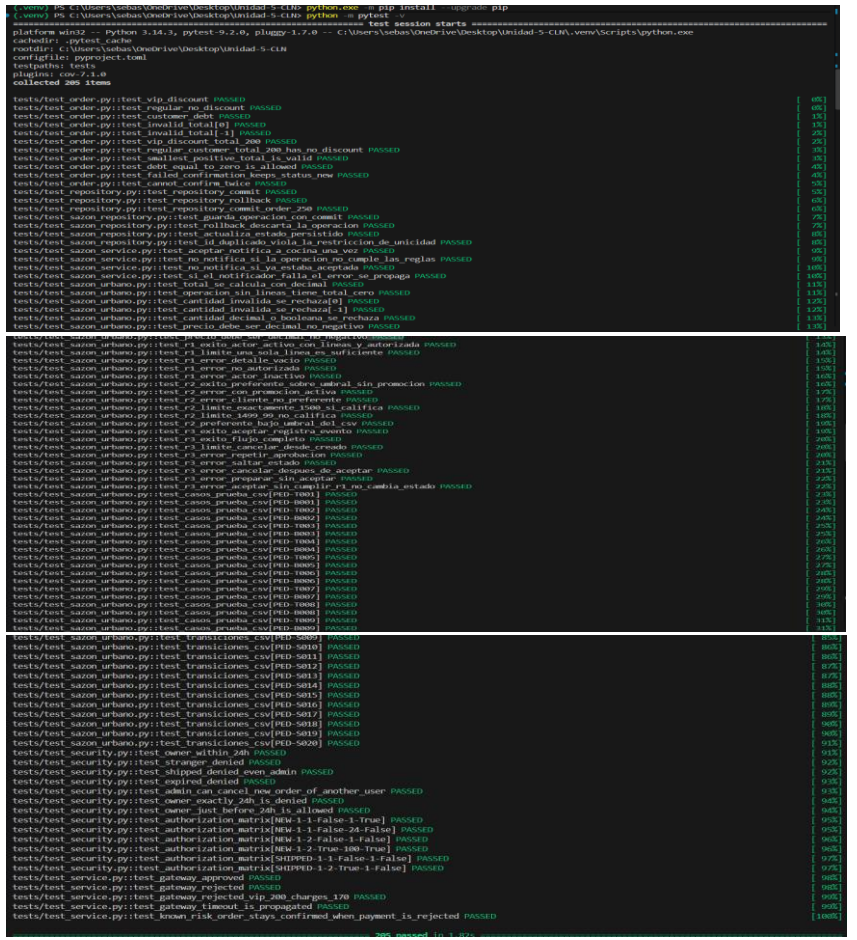
Tabla 8. Comandos de instalación y ejecución

Paso	Comando	Qué hace
1	`py -m venv .venv`	Crea el entorno virtual .venv
2	`.\.venv\Scripts\Activate.ps1`	Activa el entorno; el prompt muestra (.venv)
3	`python -m pip install -r requirements.txt`	Instala pytest, pytest-cov, SQLAlchemy y Ruff
4	`python -m pytest -v`	Ejecuta todas las pruebas; cada una debe terminar en PASSED

Tabla 9. Para qué sirven pytest y el entorno virtual

Herramienta	Explicación
pytest	Localiza las funciones cuyo nombre empieza por test_, las ejecuta y reporta cuáles pasan o fallan. Así se comprueban las reglas sin probarlas a mano.
Entorno virtual (venv)	Aísla las bibliotecas del curso para que todos usen las mismas versiones y el proyecto no dependa del Python general de la computadora.
requirements.txt	Lista las dependencias; pip las instala con un solo comando para que el resultado sea reproducible.

Tabla 10. Evidencia E1

Evidencia	Cómo obtenerla	Captura
Entorno y pruebas	python -m pytest -v	

4. E2 · Pruebas unitarias del dominio

Order.confirm() rechaza totales menores o iguales a cero, rechaza deuda pendiente, calcula el descuento VIP de 15 % y cambia el estado a CONFIRMED. Se agregaron dos pruebas de la actividad guiada.

Tabla 11. Pruebas nuevas de E2

Código

`def test_vip_discount_total_200():`
`  order = Order(200, vip=True)`
`  assert order.confirm() == 170`
`  assert order.discount == 30`
 	
`def test_regular_customer_total_200_has_no_discount():`
`  order = Order(200)`
`  assert order.confirm() == 200`
`  assert order.discount == 0`

Tabla 12. Entrada, regla, resultado esperado y resultado real

Entrada	Regla	Resultado esperado	Resultado real	Estado
Pedido 100, VIP	Descuento VIP 15 %	cobro 85, CONFIRMED	cobro 85, CONFIRMED	Cumple
Pedido 100, regular	Cliente regular sin descuento	cobro 100, CONFIRMED	cobro 100, CONFIRMED	Cumple
Pedido 200, VIP	Descuento VIP 15 %	cobro 170, CONFIRMED	cobro 170, CONFIRMED	Cumple
Pedido 200, regular	Cliente regular sin descuento	cobro 200, CONFIRMED	cobro 200, CONFIRMED	Cumple

Las pruebas siguen el patrón Arrange-Act-Assert: primero se preparan los datos (Arrange), después se ejecuta la acción que se quiere probar (Act) y al final se verifica el resultado (Assert). Los comentarios # Arrange, # Act y # Assert lo marcan en test_order.py, test_service.py y test_repository.py.

Tabla 13. Ejemplo de Arrange-Act-Assert con test_vip_discount_total_200

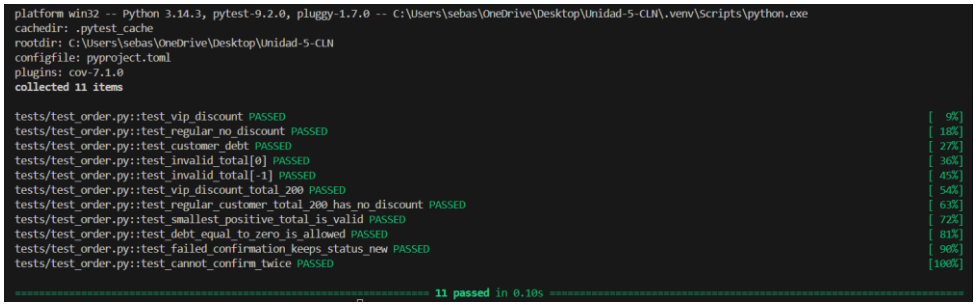
Fase	Código	Qué hace
Arrange (preparar)	order = Order(200, vip=True)	Se crea un pedido VIP de 200.
Act (ejecutar)	cobro = order.confirm()	Se confirma el pedido.
Assert (verificar)	assert cobro == 170 y assert order.discount == 30	Se comprueba el cobro con 15 % de descuento y el descuento aplicado.

Tabla 14. Casos de error y de límite del dominio

Prueba	Tipo	Qué verifica
`test_customer_debt`	Error	Con deuda pendiente se lanza CustomerHasDebt
`test_invalid_total[0] y [-1]`	Límite y error	Un total de 0 o negativo lanza ValueError
`test_smallest_positive_total_is_valid`	Límite	Un total de 0.01 se confirma
`test_debt_equal_to_zero_is_allowed`	Límite	Una deuda de 0 no bloquea la confirmación

Prueba	Tipo	Qué verifica
`test_failed_confirmation_keeps_status_new`	Error	Si la confirmación falla, el estado sigue en NEW

Tabla 15. Evidencia E2

Evidencia	Cómo obtenerla	Captura
Dos pruebas nuevas en verde	python -m pytest tests/test_order.py -v	

5. E3 · TDD: Red, Green, Refactor

El proyecto ya valida que el total sea positivo, así que repetir esa regla no produciría la fase RED. Se trabajó una regla nueva: no confirmar un pedido que ya fue confirmado.

Tabla 16. Ciclo TDD

Fase	Qué se hace	Resultado
RED	Se escribe test_cannot_confirm_twice y se ejecuta antes de tocar src/domain.py.	FAILED: Failed: DID NOT RAISE ValueError. La versión inicial permite confirmar dos veces.
GREEN	Al inicio de confirm() se agrega: if self.status == "CONFIRMED": raise ValueError("already confirmed").	PASSED. Después se ejecuta toda la suite: 205 pruebas pasan.
REFACTOR	Se podría extraer la validación a un método privado o usar una constante para el estado.	No se cambió la interfaz pública; la suite completa sigue en verde.

Tabla 17. Prueba y código de E3

Código

`# Prueba (RED)`
`def test_cannot_confirm_twice():`
`  order = Order(100)`
`  order.confirm()`
`  with pytest.raises(ValueError, match="already confirmed"):`
`  order.confirm()`
 	
`# Implementación mínima (GREEN), al inicio de Order.confirm()`
`if self.status == "CONFIRMED":`
`  raise ValueError("already confirmed")`

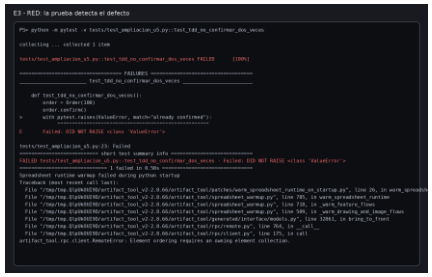
Tabla 18. Cómo reproducir RED y GREEN

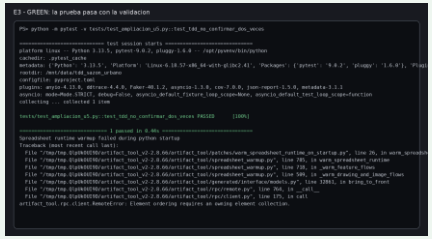
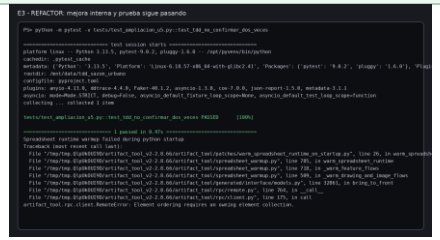
Paso	Acción
1	En src/domain.py comenta las dos líneas de la guarda de confirmación.
2	Ejecuta python -m pytest tests/test_order.py -v y captura el FAILED.
3	Descomenta las dos líneas, vuelve a ejecutar y captura el PASSED.

Tabla 19. TDD no es ejecutar pruebas después de programar

Pregunta	Respuesta
¿Qué cambia al escribir la prueba primero?	La prueba define el comportamiento esperado antes de programarlo; el RED demuestra que la prueba sí detecta la ausencia de la regla.
¿Qué pasa si la prueba se escribe después?	Si ya nació en verde, no se sabe si habría detectado el defecto; podría estar aprobando por una aserción débil.

Tabla 20. Evidencia E3

Evidencia	Cómo obtenerla	Captura
Fase RED	Prueba sin la guarda	

Evidencia	Cómo obtenerla	Captura
Fase GREEN	Prueba con la guarda	
Fase REFACTOR	Código reorganizado y pruebas conservadas en verde	

6. E4 · Base de datos y transacciones

A diferencia de una prueba de dominio, aquí interviene SQLAlchemy con una base SQLite en memoria. Cada prueba recibe su propio motor mediante la fixture engine, que lo cierra al terminar.

Tabla 21. Pruebas de persistencia

Prueba	Qué verifica
`test_repository_commit`	Se guarda un pedido de 120 y, tras commit, se consulta por id 1 con total 120.
`test_repository_rollback`	Se guarda el pedido, se hace rollback y la consulta devuelve None.
`test_repository_commit_order_250`	Prueba nueva de E4: pedido de 250 con id 2; tras commit se verifican id, total y estado NEW.

Tabla 22. Código de la prueba nueva

Código

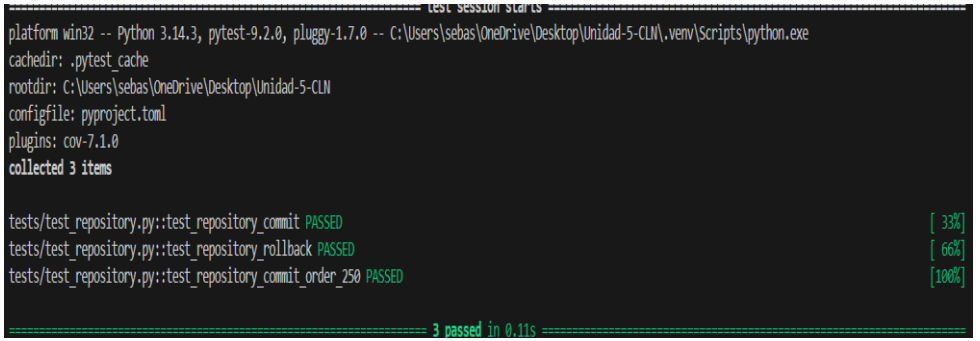
```
def test_repository_commit_order_250(engine):
    with Session(engine) as session:
        SqlOrderRepository(session).save(2, Order(250))
        session.commit()
        row = session.scalar(select(OrderRow).where(OrderRow.id == 2))
        assert row.id == 2
        assert row.total == 250
        assert row.status == "NEW"
```

Tabla 23. Commit y rollback

Operación	Qué hace	En las pruebas
commit	Hace persistente la transacción dentro de la base de prueba.	La consulta posterior encuentra el registro.
rollback	Descarta los cambios pendientes de la sesión.	La consulta posterior devuelve None.

Limitación: SQLite en memoria es rápido, pero no sustituye todas las pruebas de un motor productivo como PostgreSQL.

Tabla 24. Evidencia E4

Evidencia	Cómo obtenerla	Captura
Persistencia	python -m pytest tests/test_repository.py -v	

7. E5 · Servicios externos con mock

Un mock es un objeto controlado por la prueba que responde como si fuera la pasarela de pago. No se transmite dinero ni se hace ninguna llamada de red.

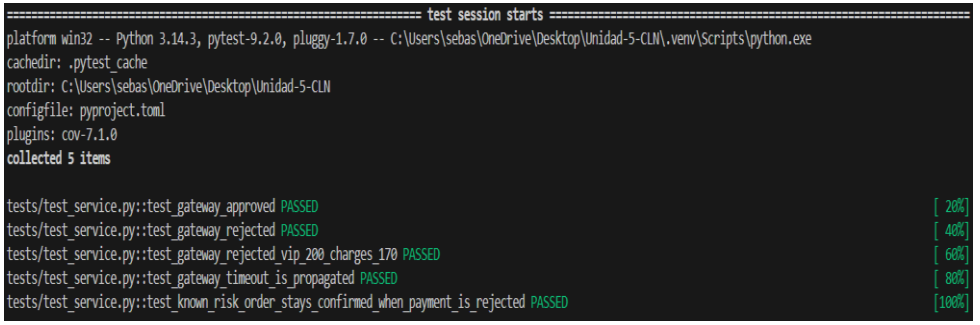
Tabla 25. Escenarios de la pasarela simulada

Escenario	Configuración del mock	Resultado esperado	Prueba
Pago aprobado, pedido de 100	charge devuelve True	checkout devuelve 100; charge se llamó una vez con 100	`test_gateway_approved`
Pago rechazado, pedido VIP de 200	charge devuelve False	RuntimeError "payment rejected"; el cobro intentado fue de 170	`test_gateway_rejected_vip_200_charges_170`
Tiempo agotado	charge lanza TimeoutError	El error se propaga; no hay reintento	`test_gateway_timeout_is_propagated`

Tabla 26. Riesgo de consistencia y propuesta de mejora

Elemento	Descripción
Riesgo	checkout confirma el pedido antes de saber si el pago fue aceptado. Si la pasarela rechaza el cobro, el pedido queda CONFIRMED sin pago.
Cómo se documenta	test_known_risk_order_stays_confirmed_when_payment_is_rejected prueba el comportamiento actual, sin ocultarlo.
Propuesta	Calcular el importe sin cambiar el estado, cobrar y confirmar solo si el cobro fue exitoso; o revertir el estado si el cobro falla. Con eventos, publicar después del commit (patrón outbox).

Tabla 27. Evidencia E5

Evidencia	Cómo obtenerla	Captura
Rutas del mock	python -m pytest tests/test_service.py -v	

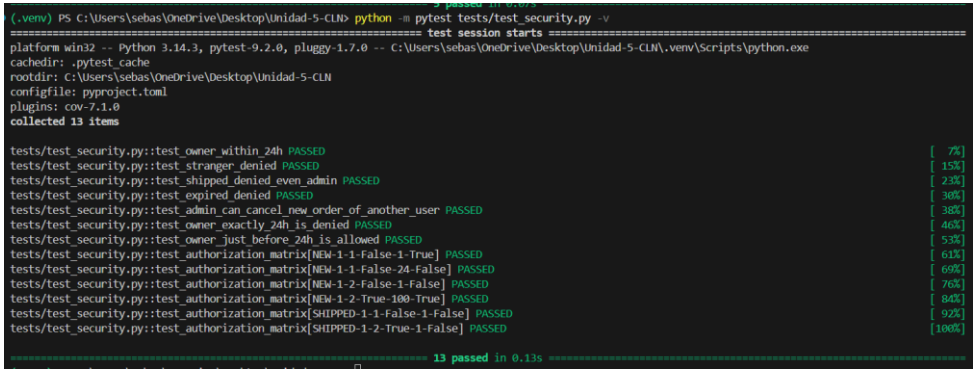
8. E6 · Seguridad y autorización

can_cancel aplica una política de cancelación: un pedido SHIPPED nunca se cancela, ni siquiera por un administrador; en los demás estados el administrador puede cancelar y el dueño solo durante las primeras 24 horas. Se prueba lo que el código realmente implementa, con fechas fijas.

Tabla 28. Matriz de autorización

Rol	Dueño del pedido	Estado	Antigüedad	Esperado	Real	Estado	Prueba
Dueño	Sí	NEW	1 h	Permitido	Permitido	Cumple	`test_authorization_matrix`
Dueño	Sí	NEW	23 h 59 min	Permitido	Permitido	Cumple	`test_owner_just_before_24h_is_allowed`
Dueño	Sí	NEW	24 h exactas	Denegado	Denegado	Cumple	`test_owner_exactly_24h_is_denied`
Tercero	No	NEW	1 h	Denegado	Denegado	Cumple	`test_stranger_denied`
Administrador	No	NEW	100 h	Permitido	Permitido	Cumple	`test_admin_can_cancel_new_order_of_another_user`
Dueño	Sí	SHIPPED	1 h	Denegado	Denegado	Cumple	`test_authorization_matrix`
Administrador	No	SHIPPED	1 h	Denegado	Denegado	Cumple	`test_shipped_denied_even_admin`

Tabla 29. Evidencia E6

Evidencia	Cómo obtenerla	Captura
Autorización	python -m pytest tests/test_security.py -v	

9. E7 · Cobertura y calidad

La cobertura del 80 % significa que al menos el 80 % de las líneas de src/ se ejecutó durante las pruebas. No significa que se detectaron el 80 % de los defectos, por eso también se comprobó que las pruebas detectan fallas reales.

Tabla 30. Cobertura por archivo (pytest-cov)

Archivo	Instrucciones	Sin cubrir	Cobertura
src\application.py	8	0	100 %
src\domain.py	18	0	100 %
src\repository.py	18	0	100 %
src\sazon_application.py	7	0	100 %
src\sazon_repository.py	24	0	100 %
src\sazon_urbano.py	62	0	100 %
src\security.py	8	0	100 %
TOTAL	145	0	100 %

Tabla 31. Comandos de calidad

Orden	Comando
1	`python -m ruff check src tests`
2	`python -m pytest -v --cov=src --cov-report=term-missing --cov-report=html --cov-fail-under=80`

Orden	Comando
3	`Abrir htmlcov/index.html en el navegador`

E7.1. Análisis estático del código con Ruff

Se utiliza Ruff para revisar estáticamente el código fuente y las pruebas del proyecto. Esta herramienta permite detectar problemas de estilo, importaciones y posibles errores comunes antes de integrar los cambios.

Comando que debe ejecutarse desde la carpeta raíz del proyecto:

```
python -m ruff check src tests
```

Resultado y evidencia: pendiente de ejecutar Ruff en el entorno del proyecto y pegar aquí la captura real de la terminal. No se registra "All checks passed!" hasta comprobarlo.

Figura E7.1. Ejecución de Ruff en la terminal de VS Code (insertar captura después de ejecutar el comando).

Tabla 32. Defectos introducidos a propósito y prueba que los detectó

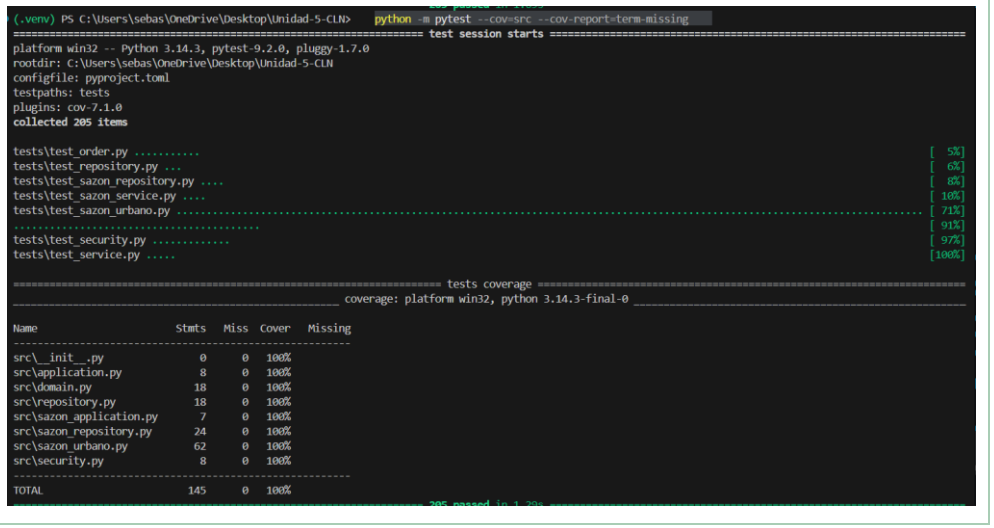
Defecto introducido	Primera prueba que falló
Descuento VIP de 15 % cambiado a 10 %	`test_vip_discount`
Total 0 permitido	`test_invalid_total[0]`
Deuda pequeña permitida	`test_customer_debt`
Permite confirmar dos veces	`test_cannot_confirm_twice`
24 h exactas permitidas	`test_owner_exactly_24h_is_denied`
Un pedido enviado se puede cancelar	`test_shipped_denied_even_admin`

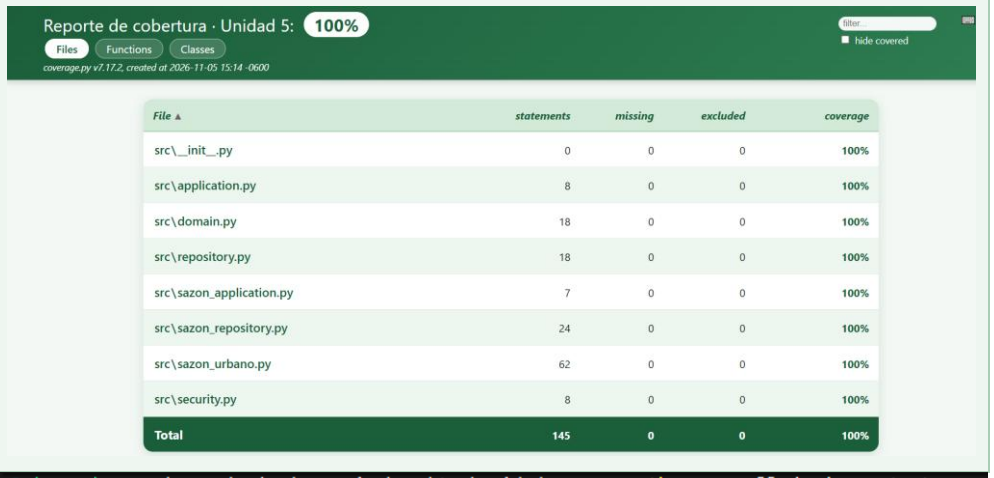
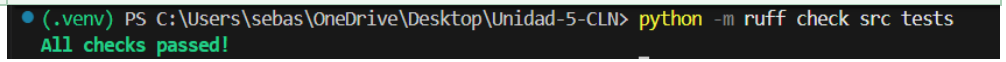
Defecto introducido	Primera prueba que falló
Se ignora el rechazo del pago	`test_gateway_approved`
Umbral de 1500 estricto	`test_r2_limite_exactamente_1500_si_califica`
Se ignora la promoción activa	`test_r2_error_con_promocion_activa`
Se permite un detalle vacío	`test_r1_error_detalle_vacio`
Se permite aceptar dos veces	`test_no_notifica_si_ya_estaba_aceptada`
No se notifica a cocina	`test_aceptar_notifica_a_cocina_una_vez`

Tabla 33. Comportamientos que la cobertura no demuestra

Situación	Explicación
can_cancel con fechas sin zona horaria	Todas las líneas se ejecutan, pero restar una fecha sin zona horaria de otra con zona lanza TypeError. No hay prueba de ese caso.
Order con un total que no es numérico	Order("100").confirm() lanza TypeError; el dominio no valida el tipo de total de forma explícita.

Tabla 34. Evidencia E7

Evidencia	Cómo obtenerla	Captura
Resumen de cobertura	python -m pytest --cov=src --cov-report=term-missing	

Evidencia	Cómo obtenerla	Captura
Reporte HTML	htmlcov/index.html	
Análisis estático con Ruff	python -m ruff check src tests	

10. E8 · GitHub Actions

El archivo .github/workflows/ci.yml hace que cada push o pull request ejecute las verificaciones sin intervención manual.

Tabla 35. Pasos del pipeline

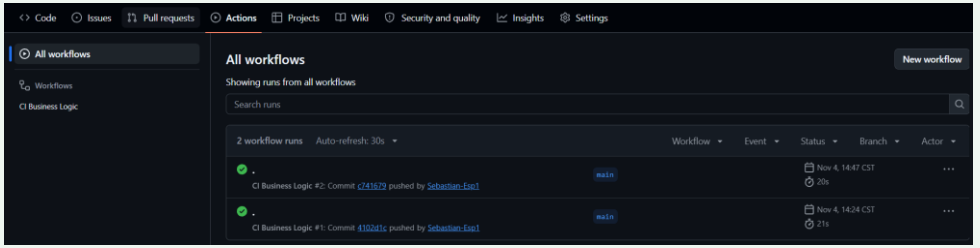
Paso	Comando	Qué verifica
1	`actions/checkout y setup-python 3.11`	Descarga el código y prepara Python
2	`pip install -r requirements.txt`	Instala las dependencias
3	`ruff check src tests`	Estilo y errores comunes
4	`pytest -v --cov=src --cov-report=term-missing --cov-fail-under=80`	Pruebas y cobertura mínima de 80 %
5	`python -m compileall -q src`	Que el código compile

Tabla 36. Procedimiento para obtener la ejecución verde, roja y verde

Paso	Acción	Resultado esperado
1	git init, git add ., git commit -m "Unidad 5: pruebas y CI", git branch -M main, git remote add origin URL, git push -u origin main	La pestaña Actions muestra el workflow CI Business Logic con marca de éxito
2	Cambiar temporalmente una aserción para que sea falsa, hacer commit y push	La ejecución falla en el paso de pytest
3	Revertir el cambio, hacer commit y push	El pipeline regresa a verde

El YAML ejecuta las verificaciones en cada push. Además, se activó una regla de protección en la rama main que exige que el check "test" del workflow pase antes de mezclar cambios; la captura de la regla está en la última fila de la Tabla 37.

Tabla 37. Evidencia E8

Evidencia	Cómo obtenerla	Captura
URL del repositorio	Pestaña Code del repositorio	https://github.com/Sebastian-Esp1/Unidad-5-CLN.git
Ejecución verde	Pestaña Actions	

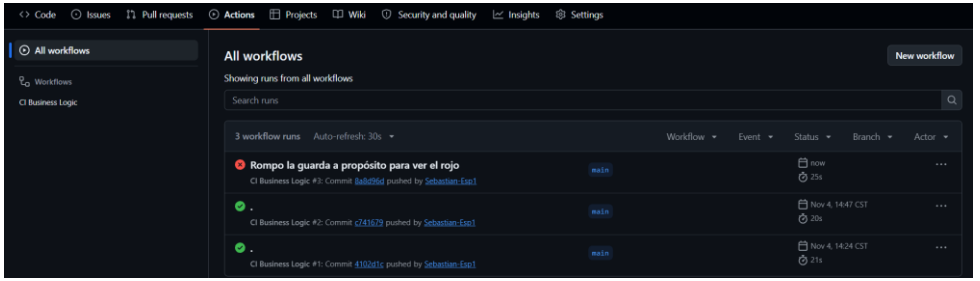
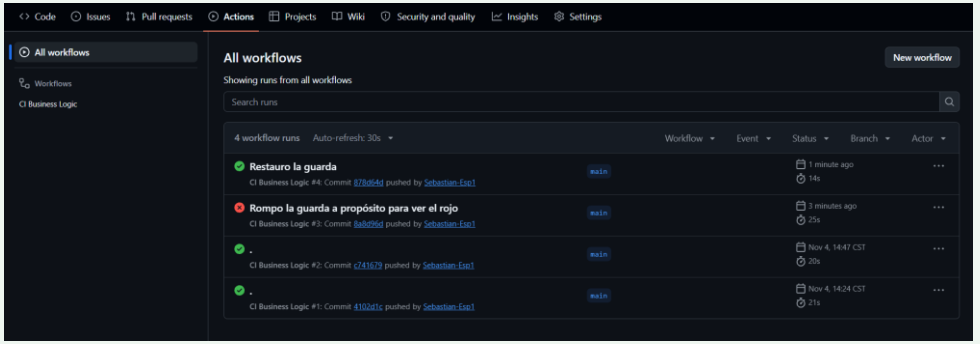
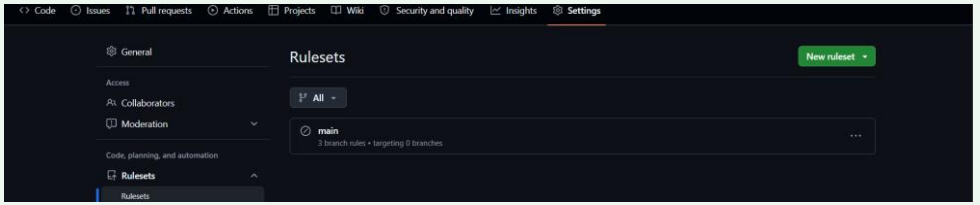
Evidencia	Cómo obtenerla	Captura
Ejecución roja	Pestaña Actions	
Ejecución verde otra vez	Pestaña Actions	
Protección de rama	Pestaña Settings > Rules > Rulesets	

Tabla 38. Qué detectó el pipeline

Pregunta	Respuesta
¿Qué detectó la ejecución roja?	Falló el paso pytest -v --cov=src --cov-report=term-missing --cov-fail-under=80. La prueba test_cannot_confirm_twice terminó en FAILED con el mensaje DID NOT RAISE <class 'ValueError'>, porque se había comentado la guarda if self.status == "CONFIRMED" en Order.confirm(). Como ese paso falló, GitHub marcó la ejecución con una X roja y los pasos siguientes no corrieron.

11. E9 · Adaptación a Sazón Urbano

Se adaptaron tres reglas del paquete de datos del PIA de Sazón Urbano (pedidos de restaurante, actor comensal, recurso platillo). Los CSV y las reglas solo se tomaron como referencia; este ejercicio es independiente del trabajo final del PIA, que debe derivarse del BRD, las historias de usuario, el BPMN y el diseño del equipo.

Tabla 39. Reglas adaptadas

Regla	Expresión	Componente
R1 puede_procesarse	Actor activo AND al menos una línea AND operación autorizada	`Operacion.puede_procesarse()`
R2 califica_beneficio	Actor preferente AND total >= 1500 MXN AND NOT promoción activa	`Operacion.califica_beneficio()`
R3 transiciones	CREADO, ACEPTADO, EN_PREPARACION, ENTREGADO; CREADO a CANCELADO; no se repite ni se salta un estado	`aceptar, preparar, entregar y cancelar`

Tabla 40. Datos del PIA usados como referencia

Archivo	Registros	Uso
`actores.csv`	40	Actor activo o preferente
`catalogo.csv`	20	Precios en MXN (Decimal)
`operaciones.csv`	60	Promoción activa y autorización
`detalle_operaciones.csv`	117	Líneas y cantidades; el total es la suma de precio por cantidad

Archivo	Registros	Uso
`casos_prueba.csv`	120	Una prueba parametrizada por caso (R1 y R2)
`transiciones_prueba.csv`	20	Una prueba parametrizada por acción sugerida (R3)

Tabla 41. Matriz de casos de Sazón Urbano

ID	Regla	Entrada	Resultado esperado	Tipo	Prueba	Resultado real
CP-01	R1 puede_procesarse	PED-O001: actor activo, 2 líneas, autorizada	True	Éxito	`test_r1_exito_actor_activo_con_lineas_y_autorizada`	True
CP-02	R1 puede_procesarse	PED-O013: sin líneas de detalle	False	Error	`test_r1_error_detalle_vacio`	False
CP-03	R1 puede_procesarse	PED-O003: exactamente 1 línea	True	Límite	`test_r1_limite_una_sola_linea_es_suficiente`	True
CP-04	R1 puede_procesarse	PED-O008: no autorizada	False	Error	`test_r1_error_no_autorizada`	False
CP-05	R1 puede_procesarse	PED-O029: actor inactivo	False	Error	`test_r1_error_actor_inactivo`	False
CP-06	R2 beneficio de referencia	PED-O023: preferente, total 6284, sin promoción	True	Éxito	`test_r2_exito_preferente_sobre_umbral_sin_promocion`	True
CP-07	R2 beneficio de referencia	PED-O018: preferente, total 6556, con promoción	False	Error	`test_r2_error_con_promocion_activa`	False
CP-08	R2 beneficio de referencia	Preferente, total exactamente 1500.00	True	Límite	`test_r2_limite_exactamente_1500_si_califica`	True

ID	Regla	Entrada	Resultado esperado	Tipo	Prueba	Resultado real
CP-09	R2 beneficio de referencia	Preferente, total 1499.99	False	Límite	`test_r2_limite_1499_99_no_califica`	False
CP-10	R3 transiciones de estado	PED-O001: aceptar desde CREADO	ACEPTADO, eventos=['OperacionAceptada']	Éxito	`test_r3_exito_aceptar_registra_evento`	ACEPTADO, eventos=['OperacionAceptada']
CP-11	R3 transiciones de estado	PED-O003: aceptar dos veces (repetir_aprobacion)	TransicionInvalida	Error	`test_r3_error_repetir_aprobacion`	TransicionInvalida
CP-12	R3 transiciones de estado	PED-O002: cancelar desde CREADO	CANCELADO	Límite	`test_r3_limite_cancelar_desde_creado`	CANCELADO
CP-13	R3 transiciones de estado	PED-O004: entregar desde CREADO (saltar_estado)	TransicionInvalida	Error	`test_r3_error_saltar_estado`	TransicionInvalida
CP-14	Persistencia (SQLite en memoria)	Guardar PED-O001 con commit	estado=CREADO, total=7426.00	Éxito	`test_guarda_operacion_con_commit`	estado=CREADO, total=7426.00
CP-15	Servicio con notificador simulado	Aceptar PED-O001 con Mock	avisar_cocina llamado 1 vez con PED-O001	Éxito	`test_aceptar_notifica_a_cocina_una_vez`	avisar_cocina llamado 1 vez con PED-O001

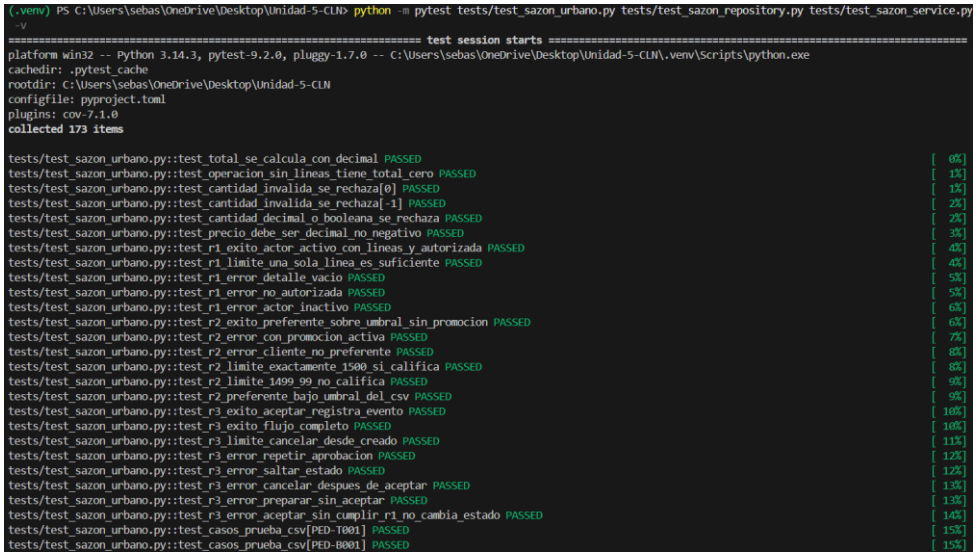
Tabla 42. Decisiones de diseño

Decisión	Motivo
Los resultados esperados se derivan de las reglas	Los CSV del alumno no traen resultados esperados. Se calcula el oráculo a partir de las columnas crudas, de forma independiente del dominio.

Decisión	Motivo
Importes con Decimal; total guardado como texto	Evita errores de punto flotante y conserva la precisión (7426.00) en la base de datos.
Acción completar interpretada como el flujo completo	El CSV no define la acción. Se asumió aceptar, preparar y entregar; es un supuesto por confirmar con el docente.
Notificador de cocina como puerto con mock	Sazón Urbano no usa una pasarela de pago. Se simula un servicio externo sin llamadas reales.
Persistencia con SQLite en memoria	Pruebas de commit, rollback, actualización de estado y unicidad del identificador.
El flujo de estados del paquete es una propuesta	No es un requerimiento original; el PIA final debe adaptarse a las reglas propias del equipo.

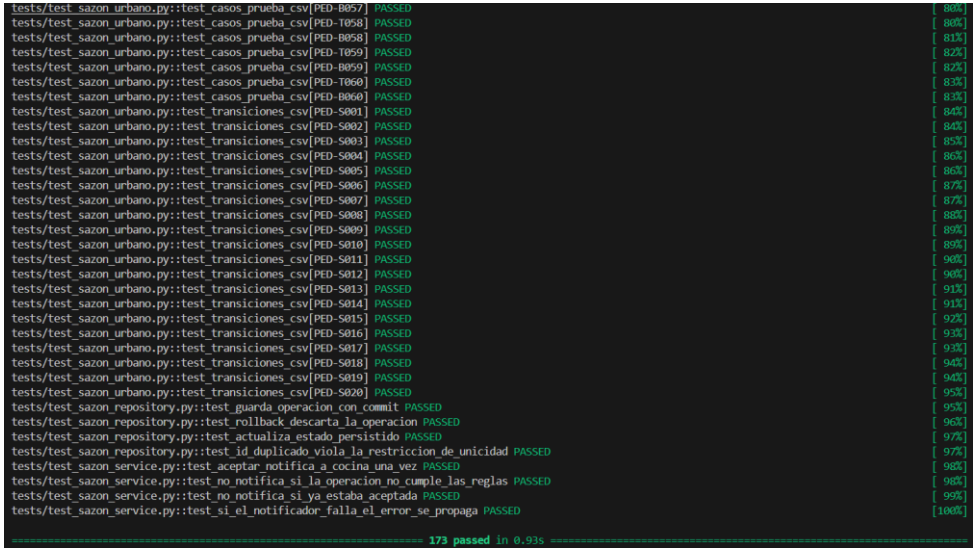
Tabla 43. Evidencia E9

Evidencia	Cómo obtenerla	Captura

Pruebas de Sazón Urbano	python -m pytest tests/test_sazon_urbano.py tests/test_sazon_repository.py tests/test_sazon_service.py -v	

```
(.venv) PS C:\Users\sebas\OneDrive\Desktop\Unidad-5-CLN> python -m pytest tests/test_sazon_urbano.py tests/test_sazon_repository.py tests/test_sazon_service.py
 -v
============================= test session starts =============================
platform win32 -- Python 3.14.3, pytest-9.2.0, pluggy-1.7.0 -- C:\Users\sebas\OneDrive\Desktop\Unidad-5-CLN\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\sebas\OneDrive\Desktop\Unidad-5-CLN
configfile: pyproject.toml
plugins: cov-7.1.0
collected 173 items

tests/test_sazon_urbano.py::test_total_se_calcula_con_decimal PASSED                                                
```

Evidencia	Cómo obtenerla	Captura
		

12. Uso responsable de la IA

La IA ayudó a revisar el proyecto y a proponer pruebas. Cada propuesta se verificó ejecutándola.

Tabla 44. Registro del uso de IA

Prompt	Resultado	Corrección o decisión	Prueba añadida
Verificar que el proyecto base funcione con los mismos comandos que el pipeline	Detectó que pytest a secas no encuentra src y que Ruff marca 12 errores.	Se agregó pythonpath y se corrigieron los imports; se confirmó ejecutando sin la carpeta actual en la ruta de Python.	Las 13 pruebas del base pasan con el comando pytest
Probar la restricción de unicidad en la base de datos	Se supuso que guardar dos objetos con el mismo id en una sesión lanzaría FlushError.	Al ejecutarlo lanzó IntegrityError. Se conservó la versión con dos sesiones, que representa un registro ya persistido.	test_id_duplicado_viola_la_restriccion_de_unicidad
Interpretar las acciones de transiciones_prueba.csv	Propuso que completar recorre todo el flujo.	El CSV no lo define; quedó como supuesto por confirmar con el docente.	test_transiciones_csv

13. Lista de verificación antes de entregar

Tabla 45. Lista de verificación

Verificación	Resultado	Cómo se comprueba
Puedo explicar por qué una prueba unitaria no requiere base de datos	Cumple	Sección 4: Order se prueba sin SQLAlchemy
Tengo evidencia real de RED y GREEN	Cumple	Sección 5: capturas FAILED y PASSED de E3
Sé diferenciar commit y rollback	Cumple	Sección 6
Demostré que el mock no genera un cargo real	Cumple	Sección 7: la pasarela es un Mock y no hay llamadas de red
Comprobé permisos y límites temporales	Cumple	Sección 8
La cobertura alcanza 80 % sin ocultar módulos	Cumple: 100 % sobre todo src/	Sección 9
El pipeline se ejecutó en GitHub y detectó un fallo	Cumple	Sección 10: ejecuciones verde, roja y verde, y protección de rama en main
Los casos del PIA representan reglas de la empresa	Cumple como referencia; el PIA final debe usar el BRD propio	Sección 11
Distingo pruebas unitarias, de integración y E2E	Cumple	Sección 2: tabla de tipos de prueba (E2E no aplica)
Aplico Arrange-Act-Assert	Cumple	Sección 4: ejemplo y comentarios en las pruebas

Tabla 46. Resultados de aprendizaje y dónde se demuestran

Resultado de aprendizaje	Dónde se demuestra
Distinguir pruebas unitarias, de integración y E2E	Sección 2: tabla de tipos de prueba
Aplicar Arrange-Act-Assert y el ciclo TDD Red-Green-Refactor	Sección 4 (Arrange-Act-Assert) y sección 5 (TDD)
Verificar persistencia y rollback de SQLAlchemy sin modificar datos de producción	Sección 6: SQLite en memoria, commit y rollback
Simular cobros con mocks, probar autorización y revisar cobertura	Secciones 7, 8 y 9
Interpretar un pipeline CI y entregar evidencia reproducible	Sección 10: Actions en verde, rojo y verde, y protección de rama

14. Reflexión del equipo

Tabla 47. Reflexión

Reflexión personal
El defecto más costoso en producción sería confirmar un pedido sin haber cobrado. En el proyecto base, OrderService.checkout() confirma el pedido antes de llamar a la pasarela de pago; si el cobro es rechazado, el pedido queda CONFIRMED y el restaurante lo prepararía sin recibir dinero. Además, una ejecución sin errores no lo detecta: el programa abre, registra y muestra resultados, pero la regla de negocio está mal. La prueba que lo evita usa un mock de la pasarela que responde False y verifica que el pedido no quede confirmado; hoy la prueba solo documenta el riesgo, y la mejora consiste en cobrar primero y confirmar después. Esta unidad nos enseñó que decir "mi programa funciona" no basta: hay que demostrarlo con casos de éxito, de error y de límite, con persistencia y servicios externos simulados, y con un pipeline que rechace los cambios que rompan esas pruebas. La cobertura ayuda, pero no demuestra corrección; por eso comprobamos también que las pruebas detectan defectos.